

React *Axios & Router*

LEXICON

Objectives

- Blocking vs Non-Blocking
- Axios
- Router

BLOCKING VS. NON-BLOCKING

- **Blocking** = waits, step by step
- **Non-blocking** = doesn't wait, keeps doing other tasks
- **JavaScript** is Single-threaded & non-blocking
- Uses callbacks, promises, async/await for async tasks
- Non-blocking keeps apps responsive while waiting for I/O
- **Async / Await in JavaScript**
 - Makes asynchronous code easier to read and write
 - Avoids "callback hell"
 - Works like synchronous code but without blocking

AXIOS

- [Axios](#) is a popular JavaScript library for making **HTTP requests** (GET, POST, PUT, DELETE, etc.).
- Works in **both the browser and Node.js**.
- Often used in **ReactJS** apps to interact with APIs (like fetching data from a server).
- Similar to the native fetch API but comes with extra features:
- Supports **interceptors** (modify requests/responses globally).
- Handles **JSON automatically**.
- Can **cancel requests**.
- Works with **timeouts**.
- Simplifies **error handling**.

Getting Started

- Axios is a [promise-based](#) HTTP Client for [node.js](#) and the browser.
- `npm install axios`
- `import axios from "axios";`

```
axios.get(`${baseUrl}/1`).then((response) => {  
  setPerson(response.data);  
});
```

```
axios.get(baseUrl).then((response) => {  
  setPersons(response.data);  
});
```

```
const savePerson = async (data) => {  
  console.log("###", data);  
  await axios.post(baseUrl, data)  
    .then((response) => {  
      // run this code  
    }).catch(e=>{  
      // run this code  
    });  
};
```

Async:

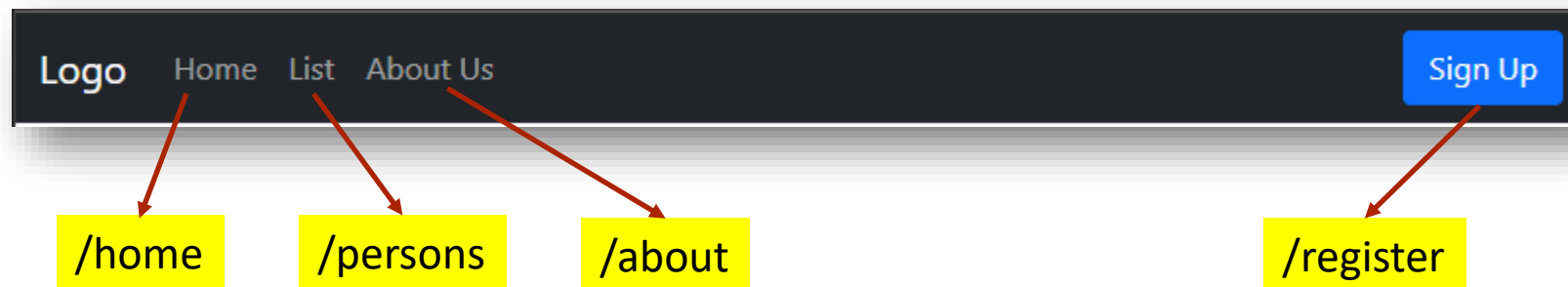
`async` keyword is used to make a function [asynchronous](#)

Await:

`await` keyword will ask the execution to wait until the defined task gets executed.

React Router

- [React Router](#) is the standard routing library for React
- A library for handling **navigation & routing** in React apps.
- To create **multiple pages** in a **Single Page Application (SPA)**.
- No full-page reloads → **faster user experience**.
- **Keeps URL & UI in sync** (back/forward works).
- Supports **dynamic routes, nested routes, and 404 pages**
- React Router DOM = **Easy navigation + Multiple pages inside one React app**



Getting Started

- React Router is a package that allow us to implement client-side routing
- `npm install react-router-dom`
- Two types of router from the React Router API ([BrowserRouter](#), [HashRouter](#))

- To import Router:

```
import {BrowserRouter, Routes, Route, Link, NavLink, Outlet } from "react-router-dom";
```

- **BrowserRouter:** Wraps your app and uses the [HTML5 history API](#) to enable client-side navigation.
- **Routes:** A container that holds and matches your different <Route>s.
- **Route:** Defines which component should render for a given URL path.
- **Link:** Lets you navigate to a route without reloading the page.
- **NavLink:** Like Link, but adds special styling when the link is active.
- **Outlet:** Placeholder that renders child routes inside a parent route.

```
// <BrowserRouter>  
http://example.com/about  
  
// <HashRouter>  
http://example.com/#/about
```

Getting Started

AppRouter.jsx: defining routes (with nested routes, index routes, and a catch-all *).

Navbar.jsx: using NavLink for navigation.

App.jsx: wrapping the app in BrowserRouter and including the navbar + routes.

```
// AppRouter.jsx
<Routes>
  <Route index path="/" element={<Home/>}/>
  <Route path="/home" element={<Home/>}/>
  <Route path="/about" element={<About/>}/>
  <Route path="/contact" element={<Content/>}/>

  {/* Dashboard with nested routes */}
  <Route path="/dashboard" element={<Dashboard/>}>
    <Route index element={<DashboardHome/>}/>
    <Route path="users" element={<ManageUsers/>}/>
    <Route path="invitations" element={<Invitations/>}/>
  </Route>

  {/* Catch-all route for undefined paths */}
  <Route path="*" element={<NotFound/>}/>
</Routes>
```

```
<main style={{flex: 1, padding: "16px"}}>
  <Outlet /> {/* content area for nested routes */}
</main>
```

```
// Navbar.jsx

<NavLink className="navbar-brand" to="/home">
  Home
</NavLink>
```

```
// App.jsx
<BrowserRouter>
  <Navbar/>
  <AppRouter/>
</BrowserRouter>
```


Hooks for Navigation & Routing

- **useNavigate** → Navigate programmatically.
- **useParams** → Access URL parameters.
- **useLocation** → Get info about the current URL.
- **useRoutes** → Define routes in a config object
- **useMatch** → Match the current URL against a specific path pattern.

Links

<https://reactjs.org/docs/hooks-intro.html>

<https://www.geeksforgeeks.org/async-await-function-in-javascript/>

<https://www.sitepoint.com/react-router-complete-guide/>

<https://reactgo.com/react-router-redirection/>